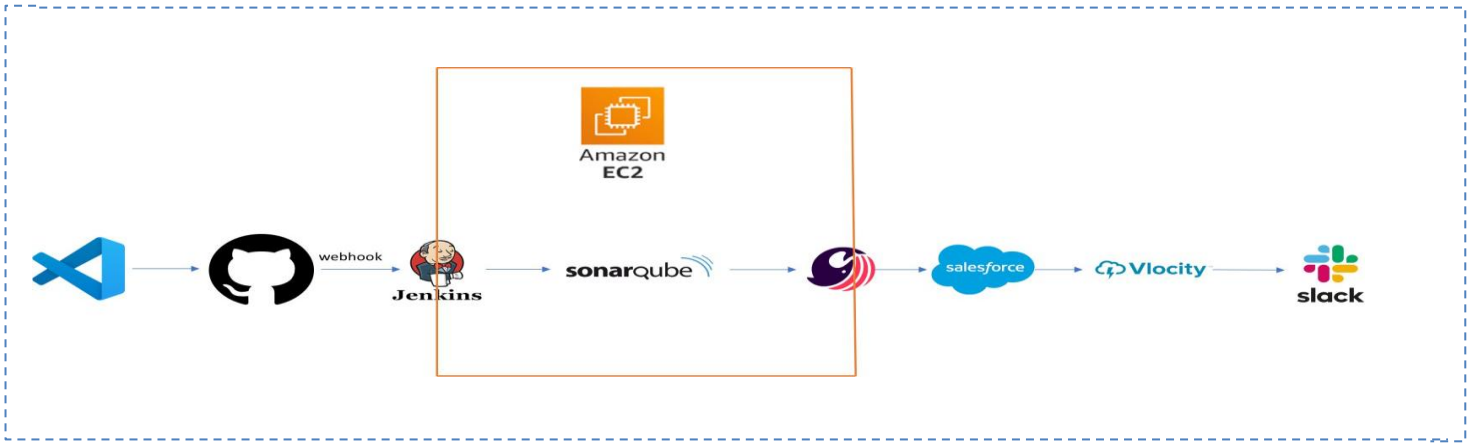
Salesforce & Vlocity CI/CD Implementation

Salesforce DevOps Live: Jenkins + Vlocity AWS Pipeline Build

Prepared by: Bharathala Ramakrishna
Role: Salesforce DevOps Engineer Date: February 2026

Architecture Overview



🚀 Project Overview

This document explains the step-by-step implementation of an end-to-end CI/CD pipeline for Salesforce and Vlocity using:

- GitHub
- Jenkins
- SonarQube
- Slack
- Amazon Web Services (EC2)

Objective

To automate:

- Code validation
- Static code analysis
- Quality gate enforcement
- Deployment to Salesforce

— without manually logging into Salesforce or triggering builds.

All tools were hosted on separate AWS EC2 instances and securely integrated using webhooks, tokens, and credential management.

1️⃣ AWS Infrastructure Setup

EC2 Instances Created

- **EC2 #1** → Jenkins Server
- **EC2 #2** → SonarQube Server

Installed on Both Servers

- Java 17 (required for Jenkins & SonarQube)
- Git
- unzip
- wget

Additional on Jenkins Server

- Salesforce CLI (sf CLI)
- Node.js
- npm
- Vlocity Build Tool
- Sonar Scanner CLI

Security Group Configuration

Port	Purpose
8080	Jenkins
9000	SonarQube
22	SSH

Elastic IP Setup

To avoid IP changes when stopping EC2:

1. AWS Console → EC2 → Elastic IPs
2. Allocate Elastic IP
3. Associate with Jenkins instance

All integrations (GitHub webhook, Sonar webhook) used:

http://ElasticIP:8080/

2️⃣ Jenkins Installation & Configuration

Installation

Installed Jenkins from official repository:

yum install jenkins -y

systemctl start jenkins

Unlocked using:

/var/lib/jenkins/secrets/initialAdminPassword

Plugins Installed

Manage Jenkins → Manage Plugins → Available Plugins

Installed:

- Git Plugin
- Pipeline Plugin
- SonarQube Scanner Plugin
- Slack Notification Plugin
- Credentials Binding Plugin

Global Tool Configuration

Path:
Manage Jenkins → Global Tool Configuration

Configured:

- JDK (Java 17)
- SonarQube Scanner
- Git

3️⃣ GitHub Integration with Jenkins

Step 1: Create Repository

On GitHub:

- Created repository
- Pushed Salesforce & Vlocity code
- Added Jenkinsfile

Step 2: Generate Personal Access Token (PAT)

GitHub Path:

Profile → Settings → Developer Settings → Personal Access Tokens → Tokens (Classic) → Generate New Token

Selected scopes:

- repo
- admin:repo_hook

Step 3: Add PAT in Jenkins

Path:
Manage Jenkins → Credentials → Global → Add Credentials

- Username: GitHub username
- Password: PAT token
- ID: github-token

Step 4: Configure Webhook

GitHub Path:

Repository → Settings → Webhooks → Add Webhook

Payload URL:

http://ElasticIP:8080/github-webhook/

Content type:

application/json

Events:

- Just the push event
- Pull requests

Now, whenever code is pushed or merged → Jenkins triggers automatically.

🔼 Salesforce Authentication Setup

Installed Salesforce CLI:

npm install -g @salesforce/cli

Authenticated interactively once:

sf org login web

Generated SFDX Auth URL:

sf org display --verbose

Copied:

Sfdx Auth Url

Stored in Jenkins

Manage Jenkins → Credentials → Global → Add Secret Text

- Secret: SFDX Auth URL
- ID: sfdx-auth

Used in Pipeline

sf org login sfdx-url --sfdx-url-file auth.txt

This allowed **non-interactive authentication** during pipeline execution.

5️⃣ Vlocity Deployment Configuration

Installed Node.js & npm:

yum install nodejs -y

Installed Vlocity Build Tool:

npm install -g vlocity

Created:

- build.properties
- job.yaml

Defined:

- packDeploy

Pipeline executes:

vlocity packDeploy -job job.yaml

Deployment logs visible in Jenkins console output.

6️⃣ SonarQube Integration

Installed SonarQube on separate EC2.

Accessed:

http://ElasticIP:9000

Step 1: Generate Token

SonarQube → My Account → Security → Generate Token

Step 2: Add in Jenkins

Manage Jenkins → Credentials → Add Secret Text

ID:

sonar-token

Step 3: Configure Sonar Server

Manage Jenkins → Configure System → SonarQube Servers

Added:

- Server Name
- Server URL
- Authentication Token

Step 4: Add Webhook

SonarQube Path:

Administration → Configuration → Webhooks → Create

URL:

http://JenkinsElasticIP:8080/sonarqube-webhook/

This enables Quality Gate enforcement.

7️⃣ Slack

Used **Slack Notification Plugin inside Jenkins**.

Steps:

1. Installed Slack Plugin
2. In Slack Workspace → Generated Incoming Webhook URL
3. Jenkins → Manage Jenkins → Configure System → Slack

Added:

- Workspace
- Default Channel
- Webhook URL

Pipeline post section:

```
post {
success {
slackSend message: "Deployment Successful"
}
failure {
slackSend message: "Deployment Failed"
}
}
```

Slack channel receives real-time updates.

8️⃣ Multi-Stage Pipeline Design

Pipeline stages:

Checkout

Build

Sonar Analysis

Quality Gate

Deploy

Post Notifications

Branch Logic:

- feature/* → Validation only
- main → Full Deployment

9️⃣ Issues Faced & Solutions:

1. Webhook returned 200 but build not triggering:

Cause: Branch filter mismatch
Fix: Corrected branch specifier in pipeline.

2. Sonar Quality Gate not waiting:

Cause: Webhook not configured
Fix: Added Sonar webhook to Jenkins.

3. IP changing after EC2 restart:

Cause: Dynamic public IP
Fix: Attached Elastic IP.

4. Salesforce authentication failures:

Cause: Expired SFDX URL
Fix: Regenerated using sf org display --verbose.